

LIBRARY MANAGEMENT SYSTEM

1.Introduction

1.1Title of the project

“LIBRARY MANAGEMENT SYSTEM”

1.2 Purpose

The MERN stack—MongoDB, Express.js, React.js, and Node.js—was used to create the web application Library Management System. This system, which replaces the conventional manual system of registers and paper forms, is intended to assist educational institutions in managing all library operations in a straightforward, quick, and entirely digital manner.

Three user types are supported by the system: administrators, librarians, and students. The administrator oversees all aspects of the system, including members, books, librarians, fees, and reports. Daily tasks including authorizing loan requests and handling book returns are handled by the librarian. Students have the ability to manage reservations, view fines, monitor their borrowed books, browse books, and request difficulties. All data are securely stored in MongoDB Atlas.

1.3 Scope

This application is designed to efficiently manage all aspects of library operations. It includes handling book information, member records, borrowing and returning transactions, fine records, and reservation data. The system supports the entire lifecycle of a library book, starting from adding it to the catalog, issuing it to a student, tracking its return, and applying fines for late returns.

The key areas addressed by this system are as follows:

- **Book Management:** The system allows for adding, editing, deleting, and viewing books, with cover images uploaded to Cloudinary.
- **User Management:** It includes student registration and login, librarian creation by an administrator, and role-based access control to ensure appropriate permissions for different user types.
- **Borrow and Return Workflow:** Students can request books, which are then approved by librarians. Students can also submit return requests, which are similarly approved by librarians.

- **Fine Management:** The system automatically calculates fines based on the return date, with customizable daily rates, a maximum fine limit, and a grace period for late returns.
- **Reservation System:** Students can reserve books that are not currently available, and they will receive notifications when copies of the book become available.
- **Reports:** Administrators can generate five types of reports, including Issued Books, Overdue Books, Fine Collection, Most Borrowed Books, and Member Activity, to track and analyze library operations.
- **Security:** The application uses JWT-based authentication, bcrypt for secure password hashing, OTP for email-based password reset, and role-based route protection to safeguard user data and system integrity.

1.4 Overview

The Library Management System is a web application designed for different user roles: Admin, Librarian, and Student. It is built using Node.js and Express.js for the backend, with MongoDB as the database, accessed through Mongoose. For security, the system uses JWT for user authentication, Cloudinary to store book cover images, Nodemailer to send OTP emails, and bcryptjs to securely hash passwords.

Login Process:

- There are separate login interfaces for Admins and Librarians, and a separate one for Students.
- Once a user successfully logs in, a JWT token is issued and stored in the browser's localStorage.
- All API routes are protected based on the user's role using authentication middleware.

Modules:

The system is organized into three main modules:

- Admin Module
- Librarian Module
- Student (User) Module

Admin Module

- **Login** – The admin logs in using an email address and password. A JWT token is generated upon successful login.

- Add Book – The admin can add a new book by providing details such as title, author, category, ISBN, description, publisher, year, number of copies, price, and uploading a cover image to Cloundinary.
- View / Edit / Delete Book – The admin has the ability to view, edit, or permanently delete books from the system.
- Add Librarian – The admin can create a new librarian account by providing the librarian's name, email, and password.
- Manage Members – The admin can view all student members, toggle their account status between active and inactive, and review individual borrowing history.
- All Reservations – The admin can view and manage all book reservations made by students.
- Fine Management – The admin can view all outstanding fines, mark fines as paid, and track the total amount collected and the amount still pending.
- Fine Configuration – The admin can set the daily fine rate, the maximum fine cap, and the grace period for late returns.
- Reports – The admin can generate five types of reports: Issued Books, Overdue Books, Fine Collection, Most Borrowed Books, and Member Activity.
- Dashboard – The admin can view statistical data and a pie chart that displays the distribution of books by category.

Librarian Module

- Login – The librarian uses their email address and password to access the system. Their role is confirmed before they can use the system.
- Issue Requests – The librarian can view all pending borrowing requests and choose to approve them. Once approved, the status of the request changes to 'Issued', and the number of available copies of the book is reduced by one.
- Books Borrowed – The librarian can see a list of all books that have been issued. This list includes the student's name, the book's title, the date the book was issued, and the due date for returning it.
- Return Requests – The librarian can view all return requests and approve them. When a return is approved, the return date is recorded, and the number of available copies of the book is increased by one.

Student (User) Module

- Register – New students can create an account by providing their name, email address, password, stream, and year of study. A unique membership ID in the format LIB-YYYYMMDD-XXXX is automatically generated.

- Login – Students log in with their email and password. Upon successful login, a JWT token is generated and stored in their browser's local storage.
- Home Page – The home page shows general library statistics and lists books organized by category.
- Browse Books – Students can search for books by title, author, or ISBN. They can also filter books by category to find what they are looking for.
- Book Details – This section provides full information about a book, including options to request it for borrowing or to reserve it if it is unavailable.
- Request Book – Students can submit a borrowing request for a book. They can only request up to four books at the same time.
- Request Return – Students can notify the librarian that they want to return a book by submitting a return request.
- My Reservations – Students can view all their active reservations for books that are currently unavailable. They can also cancel these reservations if needed.
- My Fines – Students can see a list of all fines they have accumulated. Each fine includes the number of overdue days, the amount owed, and the payment status.
- Profile – Students can view and update their personal information such as name, email, and other relevant details.
- Forgot Password – If a student forgets their password, they can reset it by requesting a 6-digit OTP (One-Time Password) to be sent to their registered email address.

Software Requirements:

- Node.js with Express.js for the backend development
- React.js version 19 along with Vite version 7 for the frontend
- MongoDB Atlas paired with Mongoose version 8.10 for database management
- Cloudinary version 1.41 together with Multer for image storage
- jsonwebtoken version 9 for handling authentication
- bcryptjs version 3 for securing passwords
- Nodemailer version 6.10 using Gmail SMTP for email and OTP services

Hardware Requirements:

- Processor of Pentium or higher
- At least 4 GB of RAM
- Minimum of 10 GB of hard disk space
- Internet connection to access cloud services

1.5 Objectives:

The development of the Library Management System aims to achieve the following:

- Replace manual and paper-based record-keeping with digital processes to manage all library activities efficiently
- Implement role-based access control to ensure that Admin, Librarian, and Student users have access only to the features relevant to their specific roles
- Automate the calculation of fines based on book return dates and daily rates that can be configured by the admin
- Maintain precise records for all book borrowings, returns, reservations, and fine payments
- Enable admins to generate comprehensive operational reports with a single click
- Allow students to check book availability, monitor borrowing status, view fines, and manage reservations from any device
- Ensure the security of user data through JWT authentication and bcrypt password hashing

1.6 Existing system:

In the current system, library records are kept manually through physical registers, printed cards, and spreadsheets. Students complete paper forms to borrow books, and librarians manually record the dates books are issued and calculate due dates. Fines are calculated by hand, which can lead to disputes. There is no automated method for students to check if a book is available or to place a reservation. Reports are compiled manually from several registers.

Disadvantages:

- Maintaining borrowing records manually is time-consuming and prone to errors.
- Physical documents such as registers and cards can be lost or damaged.
- Manual fine calculations result in inconsistencies and disputes.
- Locating a student's borrowing history or book availability requires going through physical registers.
- Creating summary reports involves extensive manual work and is often inaccurate.
- There is no way for students to reserve books that are currently unavailable.
- There is no security — anyone with access to the register can view or modify records.

1.7 Proposed System

The proposed Library Management System is a full-stack web application that tackles all the limitations of the manual system. By moving library operations to a cloud-based digital platform, the system ensures speed, accuracy, and accessibility from any device with a web browser.

Advantages of the proposed system:

- All data is securely stored in MongoDB Atlas, eliminating the risk of data loss.
- Fine calculation is fully automated based on configurable rules set by the admin, ensuring consistency.
- Students can check book availability, request books, and view their records from any device at any time.
- Librarians can quickly approve requests and process returns through a simple interface.
- Admins can generate detailed reports with a single click.
- The reservation feature allows students to automatically queue for unavailable books.
- Role-based security ensures that each user can only access features relevant to their role.
- OTP-based password reset via email ensures secure account recovery.

2. System Analysis

System Analysis is the process of studying and understanding the current problem and the requirements of the new system. It involves identifying shortcomings in existing processes, collecting requirements, assessing the system's feasibility, and planning the steps necessary before development starts. The main purpose of system analysis is to enhance the efficiency of the current system.

2.1 Problem Statement

In the current library setup, all records are kept manually. Borrowing records are written in physical registers, fine amounts are calculated by hand, and there is no automated way for students to check book availability or reserve books. The lack of a central system makes it hard to produce accurate reports or track individual student activity. Searching for records is slow, and there is a constant risk of data loss due to physical damage. The proposed Library Management System solves these issues by providing a fully digital, role-based, and cloud-supported platform that can be accessed through any web browser.

2.2 Feasibility Study

A feasibility study is a way to test a system proposal based on its practicality and impact on the organization. It is the most common method used to evaluate a proposed system. The feasibility study for the Library Management System looks at four main areas:

2.2.1 Economic Feasibility

Economic feasibility checks if the benefits of the system are worth the cost of developing it. The Library Management System is built entirely using free and open-source technologies — React.js, Node.js, Express.js, and MongoDB. The frontend is deployed on Vercel (free tier), the database is hosted on MongoDB Atlas (free tier), and Cloudinary is used for image storage on a free plan. There are no software licensing costs. Therefore, this project is economically feasible.

2.2.2 Operational Feasibility

Operational feasibility checks if the system will function well once it's installed and if it meets user needs. The Library Management System is designed to be simple and easy to use. Each user type only sees the features that are relevant to their role, which reduces confusion. Toast notifications clearly show any errors or success messages. With basic training, any staff member or student can use the system easily. Therefore, this project is operationally feasible.

2.2.3 Technical Feasibility

Technical feasibility evaluates if the needed technology, hardware, and software are available to build the system.

Node.js and Express.js provide a reliable and scalable backend. MongoDB is flexible and well-suited for the semi-structured data used in a library system. React.js allows for fast and responsive UI development. Libraries such as JWT, bcryptjs, Mongoose, Multer, Cloudinary, and Nodemailer are all stable and regularly updated. The development team has the necessary skills. Therefore, this project is technically feasible.

2.2.4 Implementation Feasibility

Implementation feasibility checks if the system can be deployed within the current environment without causing disruptions. The following questions were considered:

- Can the application be installed within the current environment? Yes — the frontend is a static React app hosted on Vercel and the backend runs on any cloud server.
- Will management and users support the installation? Yes — the system is simple and requires only a web browser on the client side.
- Will the proposed system cause any harm to operations? No — it runs independently and does not interfere with any other institutional software.

Therefore, this project is feasible for implementation.

2.3 Project Planning

The Library Management System was developed by a team of four members working collaboratively using GitHub. The project was divided into four main areas: backend development, frontend development, database design, and testing.

Work was evenly distributed among team members, with each person responsible for specific modules. Regular GitHub commits and pull requests were used to monitor progress, review code, and integrate contributions. The project followed an iterative approach, with features being developed, tested, and refined in cycles.

2.4 Project Scheduling

The project was completed over a period of approximately four months. The following table illustrates how the work was allocated across weeks and months.

	Week 1	Week 2	Week 3	Week 4
1 st Month	Requirements gathering, tech stack selection, GitHub setup	Database schema design for all 8 MongoDB collections	Backend setup, user authentication with JWT and bcrypts	Fronted setup with React.js and Vite
2 nd Month	Book management API and UI	Admin, Librarian, Student role logic	Borrow and return workflow	Fine calculation and configuration module
3 rd Month	Reservation system	Report generation (5 types)	OTP-based password reset with Nodemailer	Integration testing of all modules
4 th Month	UI polishing, Framer Motion animations	Deployment on Vercel and cloud server	Report writing and documentation	Final testing and project submission

Table 2.4.1 Project Scheduling

2.5 Activity Chart

Week Activity	January				February				March				April			
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Requirement Analysis																
Database Design																
Backend development																
Frontend development																
Integration & Testing																
Deployment																
Documentation																
Final Submission																

Table 2.5.1 Activity Chart

2.6 Software Engineering Paradigm (Process Model)

Agile Model

The Library Management System was developed using the Agile software development model. In this approach, software is built in small, iterative cycles. At the end of each cycle, a functional part of the system is created, reviewed, and refined before proceeding to the next feature. This model was selected because the project requirements changed as development continued, and the team needed the flexibility to introduce new features gradually. Features such as the reservation system, fine configuration, and report generation were added during separate iterations. The team worked on different modules simultaneously and integrated their work through GitHub.

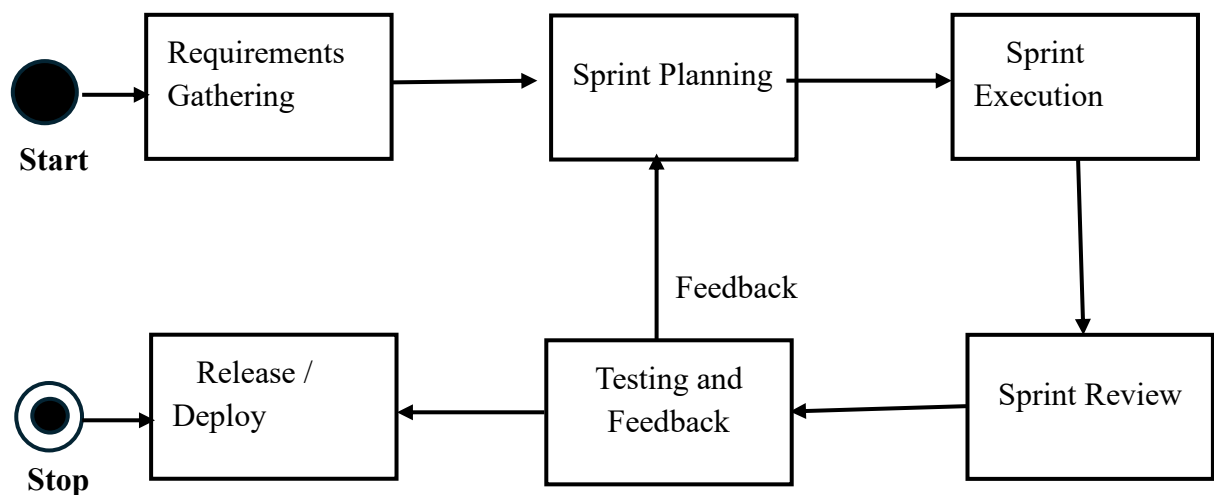


Figure 2.6.1 Agile model

Advantages of the Agile Model:

- Features are created and checked in short cycles, which makes it easier to find and fix problems early on.
- The team can change their approach based on new requirements without stopping the whole project.
- At every stage of the project, there is a working version of the product, which makes it simple to show progress.

- Team members can work on different parts of the project at the same time and combine their work using version control.

Disadvantages of the Agile Model:

- It needs regular communication and teamwork among everyone involved.
- The project might grow larger than planned if new features are added without careful planning.
- If documentation isn't kept up to date along with the development work, it may become outdated.

2.7 System Diagrams

2.7.1 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a visual way to show how a system or part of a system works. It includes data flows, processes, sources, sinks, and data stores, all shown through simple symbols. DFD is one of the most important tools in system design. It helps to understand the system, how it interacts with other components, and how data moves through the system.

DFD shows how information flows and changes as it goes through different steps. It is a visual method that illustrates how data travels from input to output. DFD is also called a bubble chart or Data Flow Graph. At any level of detail, a DFD can be used to represent the system.

There are four types of system components:

✓ Data Flow

In a system, data moves in a particular direction from its starting point to its ending point. A data flow is a group of data that shows how data moves within the system. Every data flow should have a label to explain what data is being moved. An arrow pointing towards a process shows input, and an arrow pointing away shows output.

✓ Process

Processes change incoming data into outgoing data. They are actions taken on data by people, machines, or computers. The data leaving a process is always different from the one entering it. Each process must have a clear and meaningful name.

✓ External Entities

External entities are organizations, departments, or individuals that act as a source or destination of data. These entities exist outside the system's boundaries. In this system, the external entities are Admin, Librarian, and Student.

✓ Data Store

A data store is a place within the system where data is stored. A data store must have at least one data flow going into it or out of it. It must be labeled with a name between two parallel lines. In this system, the data stores are MongoDB collections such as Users, Books, BorrowedBooks, Fines, and Reservations.

DFD Symbols:

Symbols	Description
	Source or Destination (External Entity)
	Data flow
	Process
	Data Store

Table 2.7.1 Data flow Diagram Symbols

Context Flow Diagram (Level 0 DFD):

The context diagram illustrates the Library Management System as a single process. There are three external entities: Admin, Librarian, and Student. The Admin issues management instructions and gets reports in return. The Librarian provides approval decisions and handles borrow and return requests. Students submit borrow and reservation requests and receive updates about books and fines. All the information is kept in a cloud-based database.

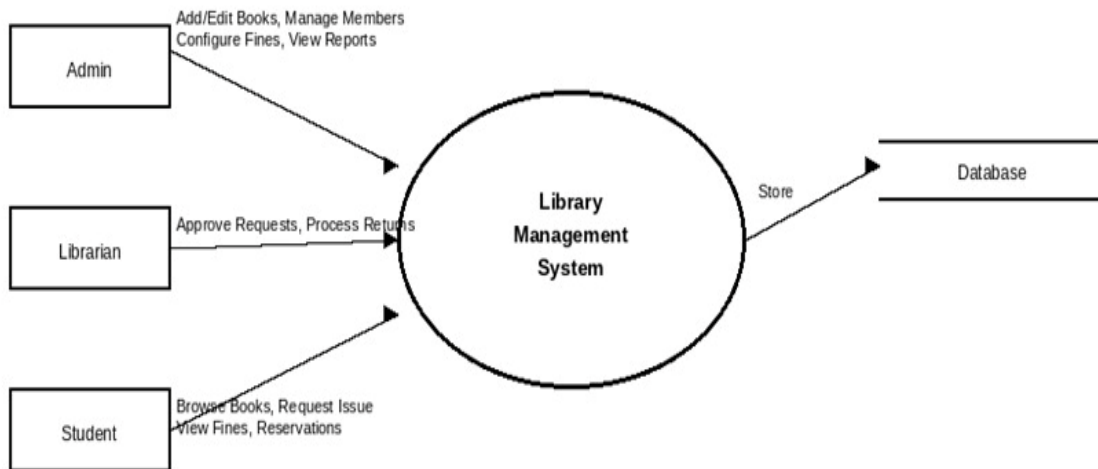


Figure 2.7.1 Level 0 DFD (Context Diagram)

Level 1 DFD for Admin:

At Level 1, the admin interacts with processes such as Login and Authentication, Book Management, Librarian Management, Member Management, Fine Configuration, Fine Management, Reservation Viewing, and Report Generation. Each of these processes involves reading from or writing to specific data stores including Books, Users, Fines, FineConfig, Reservations, and Reports.

Level 1 DFD for Admin

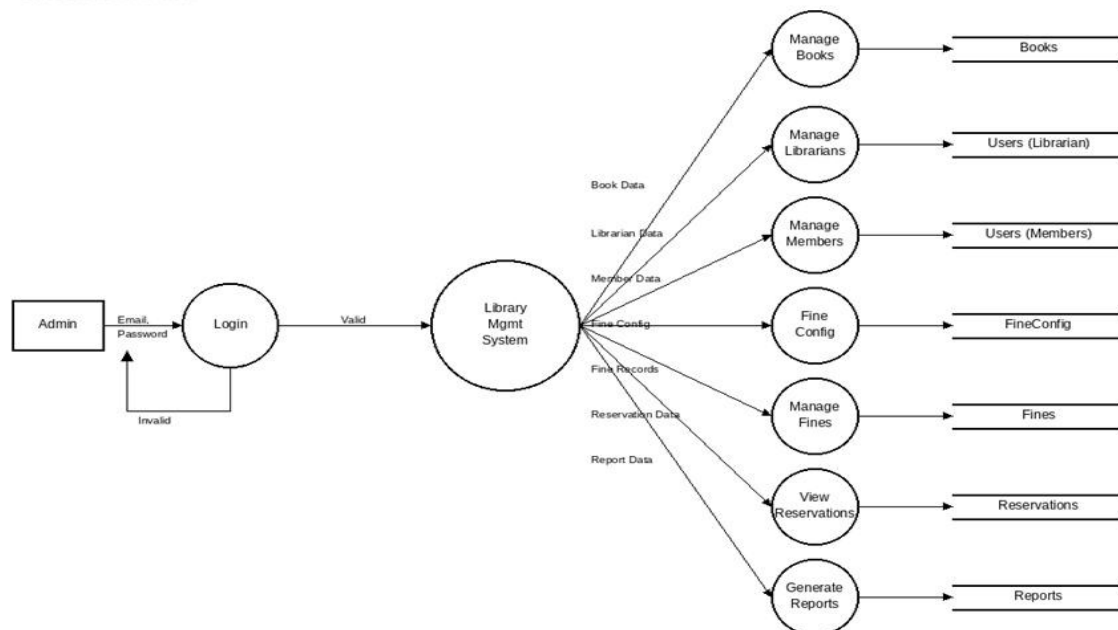


Figure 2.7.3 Level 1.1 DFD for Admin

Figure 2.7.2 Level 1.1 dfd for Admin

Level 1 DFD for student:

At Level 1, the Student engages with processes such as Registration and Login, Browse and Search Books, Request Book Issue, Request Book Return, View Fines, Manage Reservations, and Update Profile. These processes are connected to the Users, Books, BorrowedBooks, Fines, and Reservations data stores.

Level 1 DFD for Admin

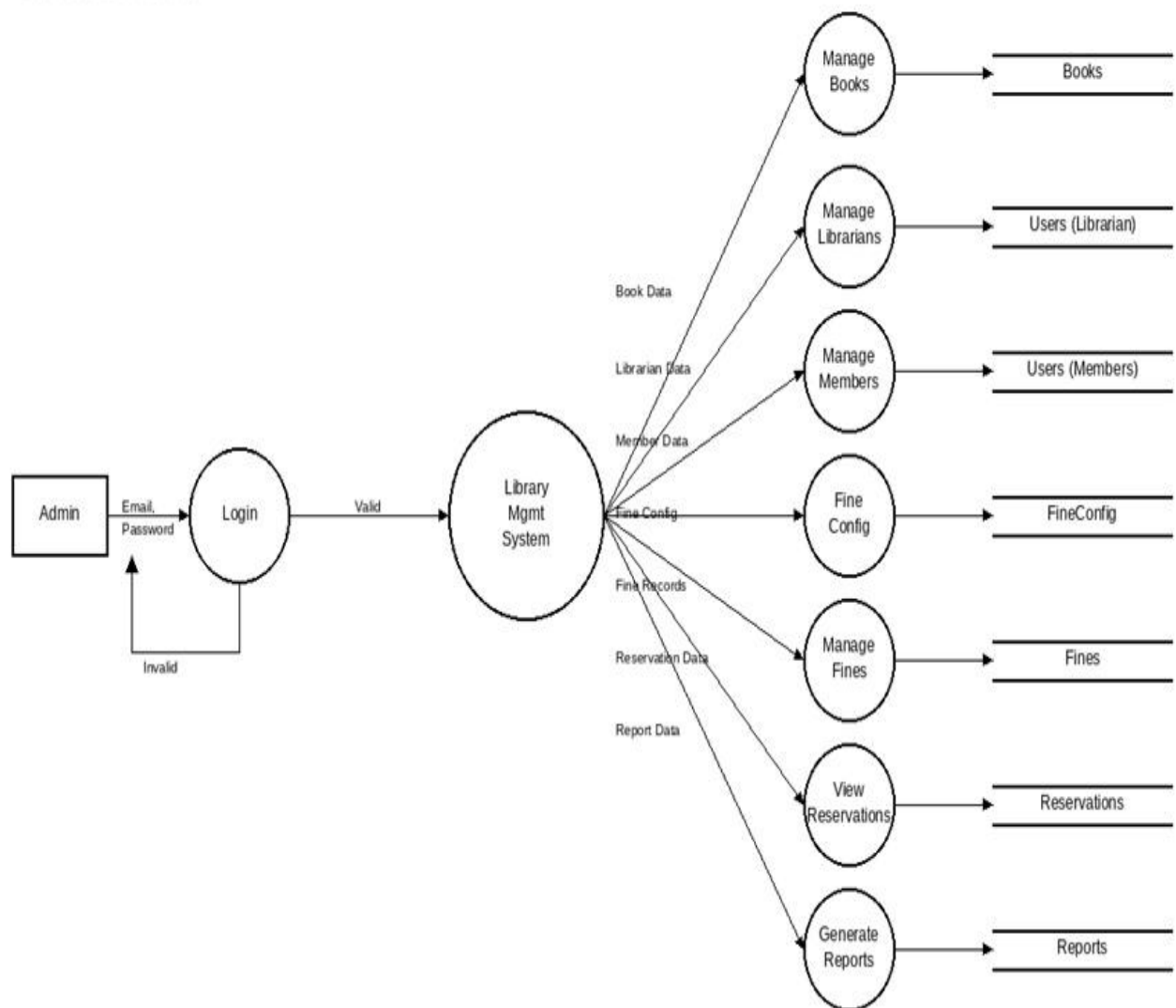


Figure 2.7.3 Level 1.1 DFD for Admin

2.7.2 Entity Relationship Diagram (ER Diagram)

An Entity Relationship (ER) Diagram is a visual tool that illustrates the logical structure of a database. It displays the entities within a system, their attributes, and the relationships connecting them. In ER diagrams, entities are represented by rectangles, attributes by ellipses, and relationships by diamonds. Each entity must have a primary key — a set of attributes that uniquely identifies each record.

Name	Notation	Description
Entity		Represents a real-world object. Shows as a Rectangle
Attribute		Property of an entity. Shown as an Ellipse.
Relationship		Association between entities. Shown as an Diamond.
Link		Connects attributes to entities and relationships.
Key Attribute		Unique identifying attributes. Shown as underlined ellipse.
Multivalued Attribute		Attribute with multiple values. Shown as double ellipse.
Cardinality	1:1 1:M M:M	1:1, 1:M, M:1, M:M – max relationships an entity can have

Table 2.7.2 Entity Relationship diagram Notation

Key Entities and Relationships in the Library Management System:

- **User** — name, email, password, role (admin/librarian/user), status, membershipId, stream, year. One user can borrow many books (1:M with BorrowedBooks).
- **Book** — title, author, category, ISBN, description, publisher, publicationYear, totalCopies, availableCopies, price, coverImage. One book can appear in many borrow records (1:M).

- **BorrowedBooks** — Links User and Book with issueDate, dueDate, returnDate, fineAmount, status, approvedBy. Status lifecycle: Requested → Issued → Requested Return → Returned.
- **Fine** — Links User, Book, and BorrowedBooks with amount, paidStatus, paidAt, daysOverdue, ratePerDay.
- **Reservation** — Links User and Book with reservationDate, expiryDate, status, notifiedAt.
- **FineConfig** — ratePerDay, maxFineCap, gracePeriod. Updated by Admin only.
- **OTP** — email, otp, createdAt. Expires after 10 minutes. Used for password reset.
- **Contact** — name, email, subject, message, date. Stores Contact Us form submissions.

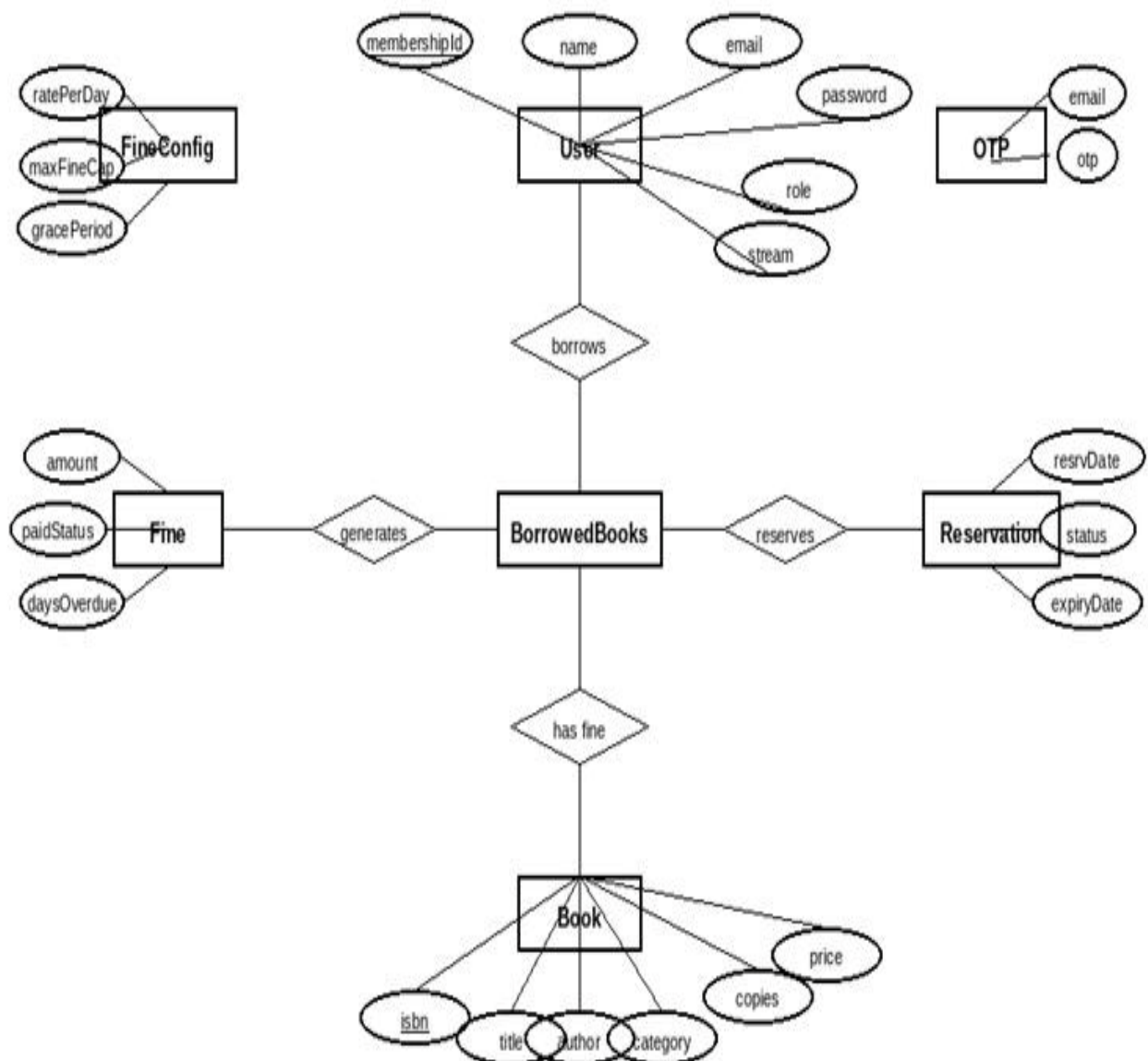


Figure 2.7.6 Entity Relationship Diagram

2.7.3 Activity Diagram

Activity diagrams are linked to program flow plans, such as flowcharts, and are used to display the activities that occur within a system. They illustrate the sequence of actions carried out by each type of user, starting from login to logout. In these diagrams, a filled circle signifies the beginning, a bullseye circle represents the end, rectangles denote activities, and a diamond indicates a decision point.

Activity Diagram for Admin:

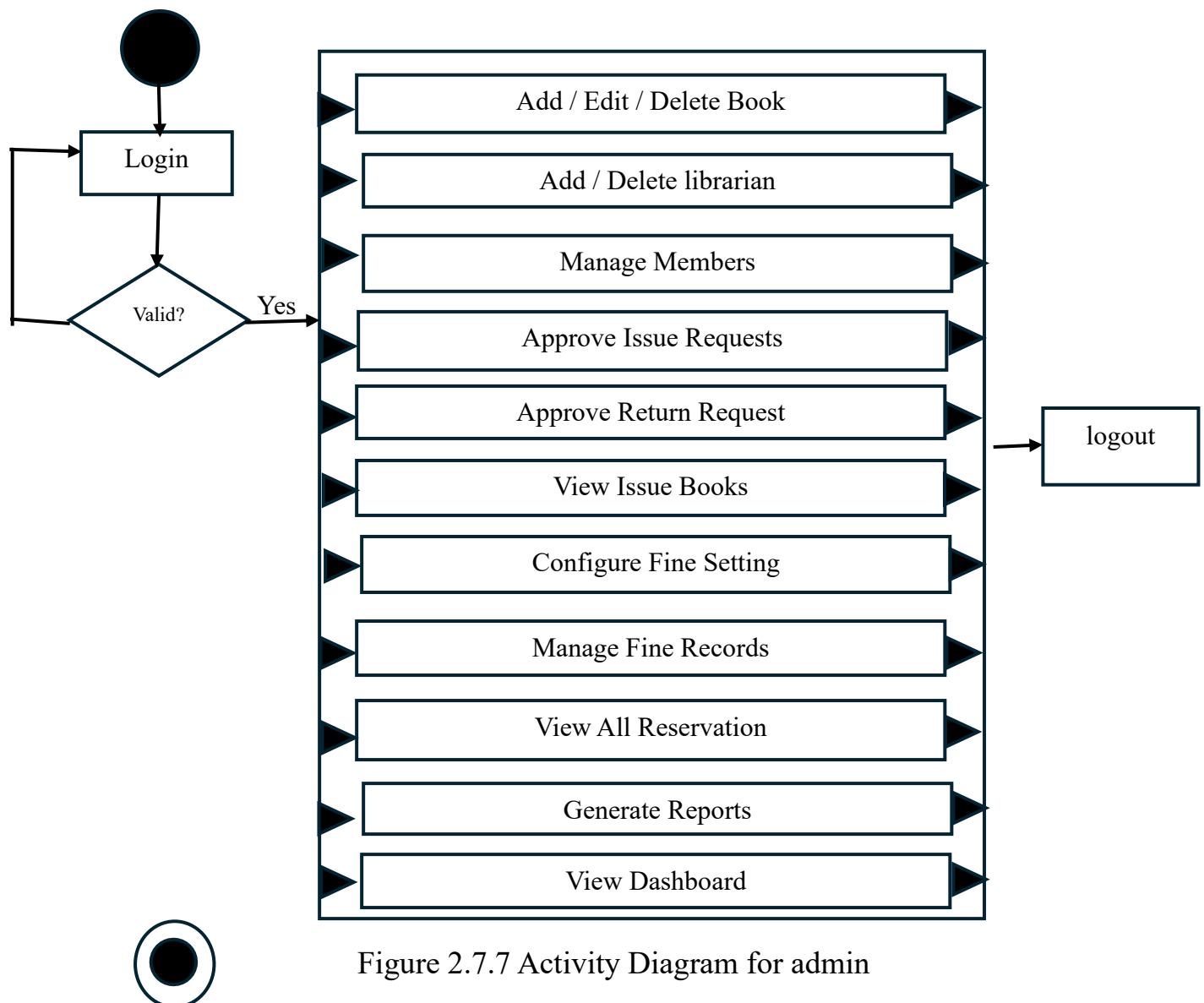


Figure 2.7.7 Activity Diagram for admin

Activity Diagram for Librarian:

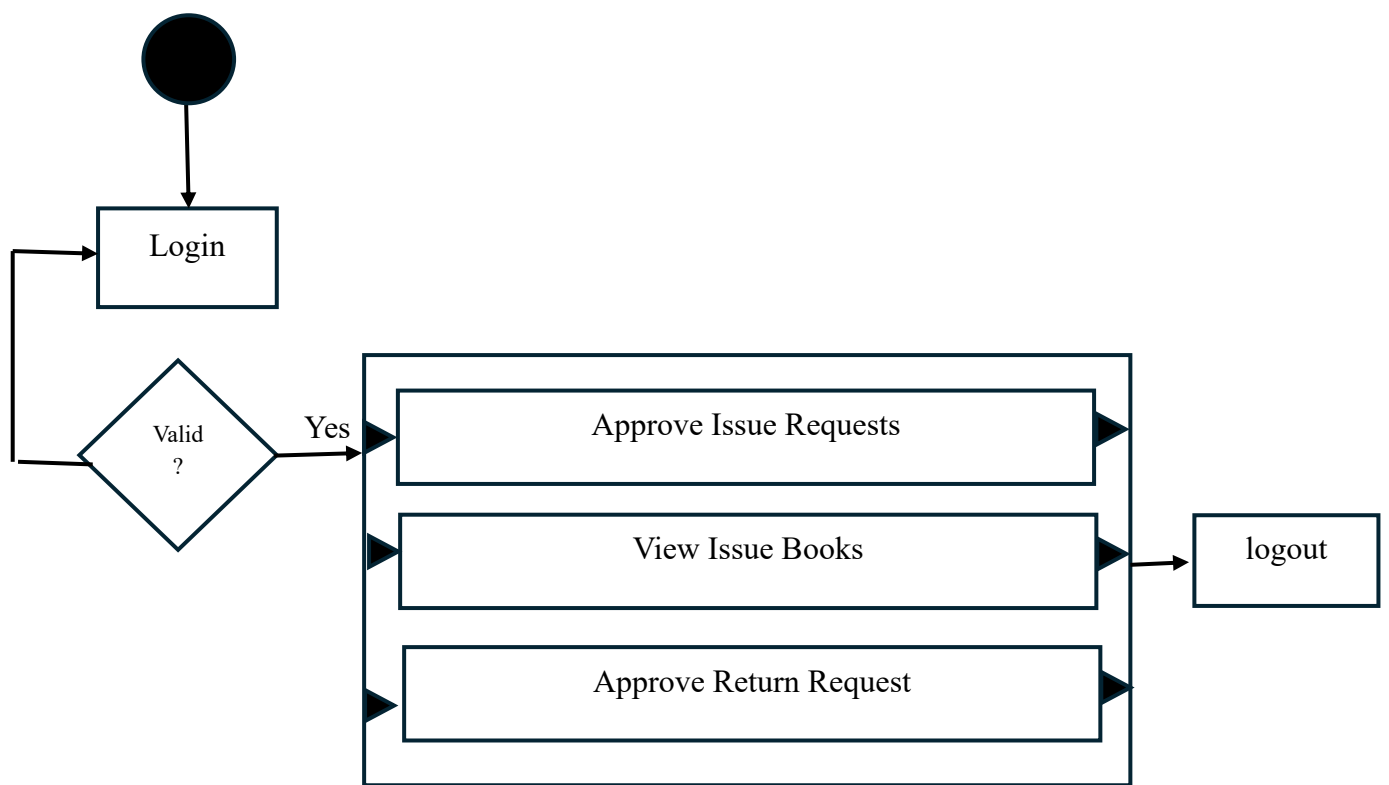


Figure 2.7.8 Activity Diagram for librarian

Activity Diagram For Student:

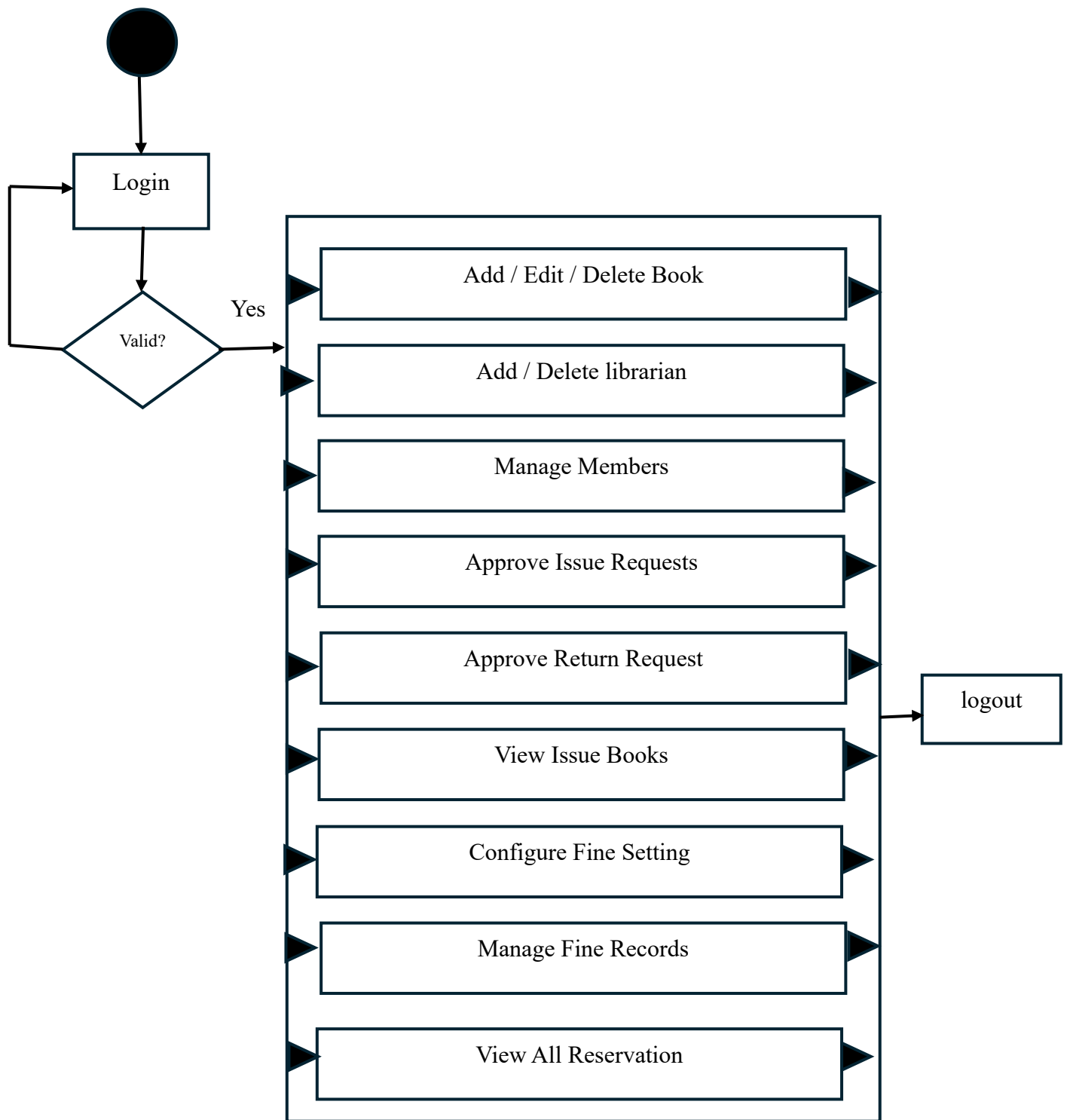


Figure 2.7.8 Activity Diagram for student



2.7.4 Use Case Model

Use Case Diagrams depict the system's functionality from the perspective of the user. Use cases are employed during the requirements gathering and analysis phases to represent

the system's features. They describe how the system behaves from an external viewpoint, highlighting the different types of users (actors) and the actions they can perform.

Shape	Purpose
	Defines the boundary of the system
	Defines the Use Case
	Defines an Actor with respect to the system
	Actor-Use case interaction
<<include>>	Include is a Directed relationship between two use cases
<<extends>>	Is a generalisation relation where an extending use case continues the behaviour of a base use case

Use Case Diagram for Librarian

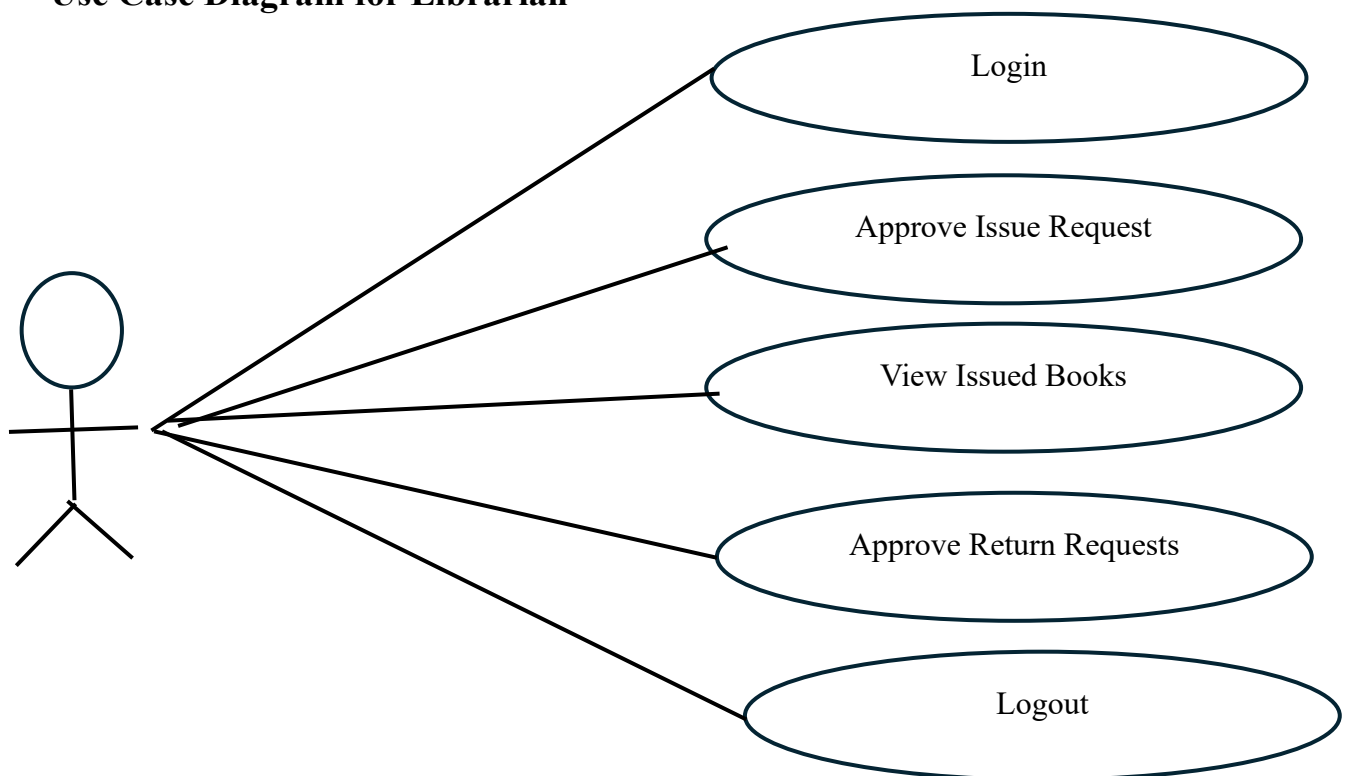


Figure 2.7.9 Use Case Diagram for Librarian

Use Case Diagram for Admin

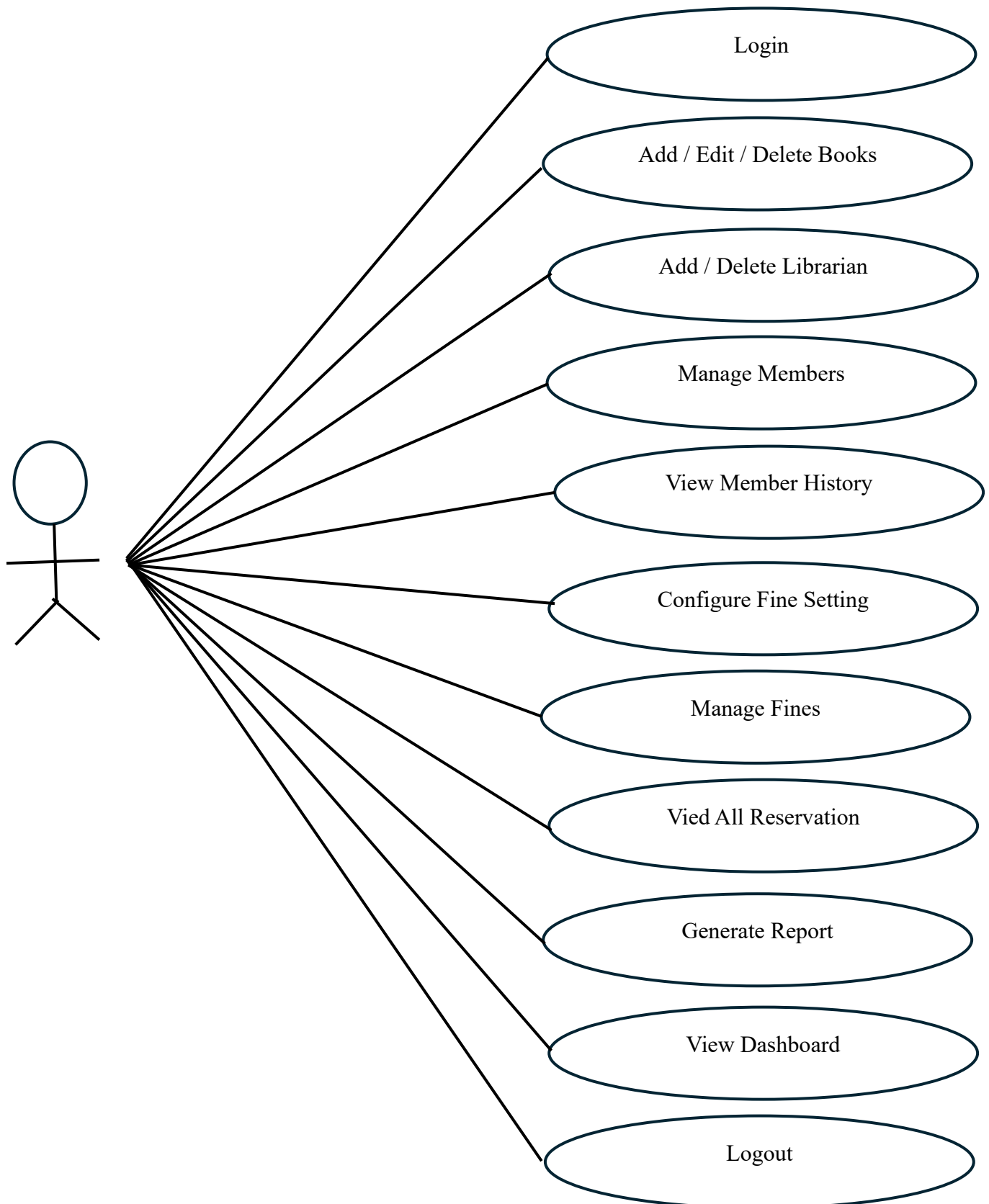


Figure 2.7.10 Use Case Diagram for Admin

Use Case Diagram for Student

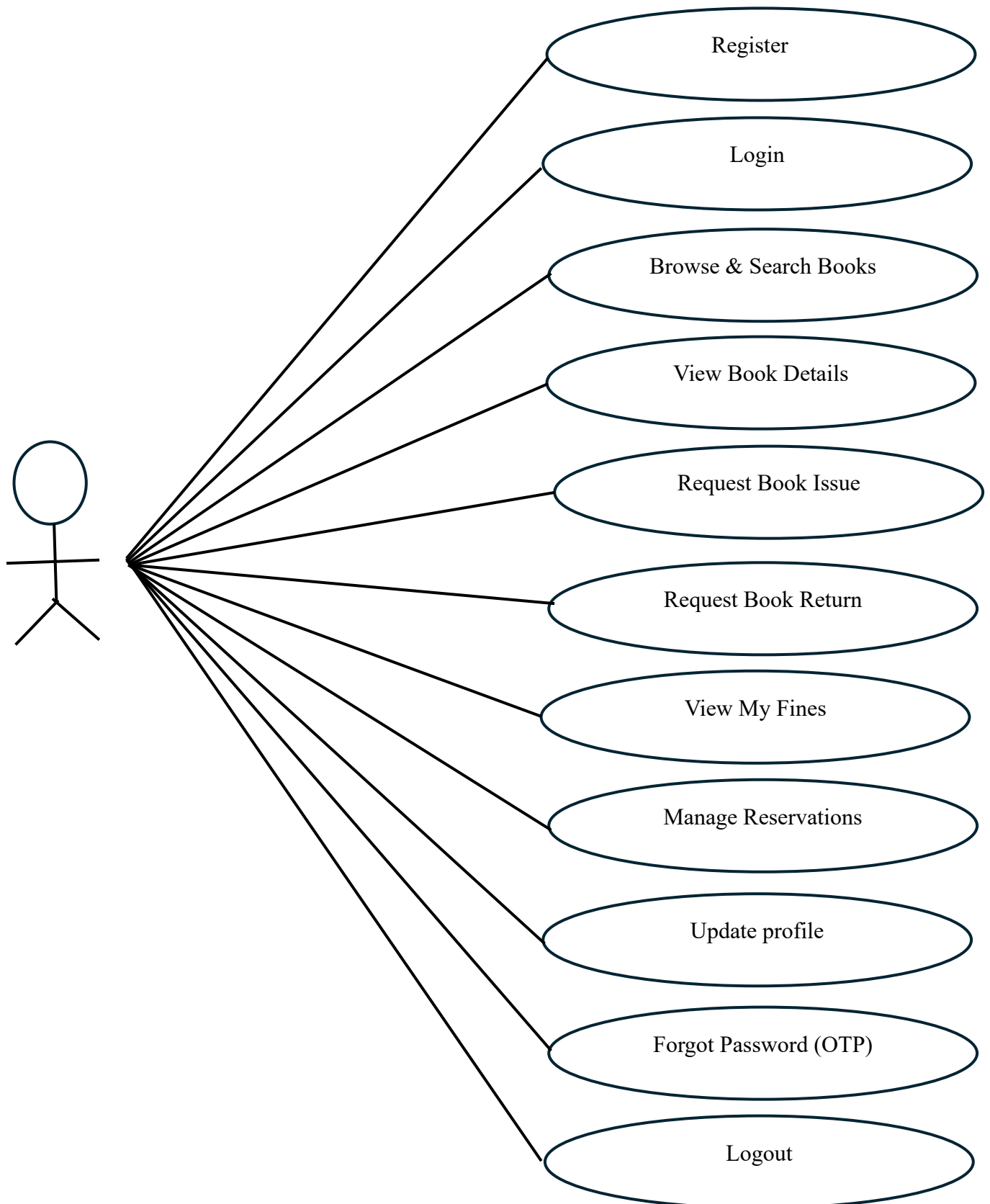


Figure 2.7.11 Use Case Diagram for Student

2.8 Software Requirement Specification (SRS)

2.8.1 Introduction

A Software Requirement Specification (SRS) is a document that thoroughly explains how a system should behave and what it needs to do. Its main goal is to help communication between the client, users, and developers. A well-prepared SRS ensures that the development team clearly understands the system's requirements and gives a clear reference for checking if the final product meets those needs. A good SRS also helps to reduce mistakes and lower the overall cost of developing the product.

2.8.1.1 Purpose

The purpose of this SRS is to outline the specific needs for the Library Management System. This system is a full-stack MERN web application designed for educational institutions to manage library activities digitally. Its primary goal is to handle all book-related tasks, such as adding books, lending books to students, tracking returns, calculating fines, managing reservations, and creating reports.

2.8.1.2 Scope

The system's scope is to provide an easy-to-use web interface for Admin, Librarian, and Student users. It manages book availability, records all borrow and return activities, automatically calculates fines, handles reservations, and produces reports. It also allows for retrieving member information, viewing borrowing history, and generating administrative summaries at any time from any device.

2.8.1.3 Definitions, Acronyms and Abbreviations

Term	Full Form / Defination
MERN	MongoDB, Express.js, Node.js
API	Application Programming Interface
JWT	JSON Web Token – used for user authentication
OTP	One-Time Password – used for password reset via email
ISBN	International Standard Book Number

UI	User Interface
DB	Database
CRUD	Create, Read, Update, Delete
REST	Representational state Transfer – architecture style for APIs
DFD	Data Flow Diagram
SRS	Software Requirement Specification
RAM	Random Access Memory

Table 2.8.1 Definitions, Acronyms and Abbreviation

2.8.1.4References

- <https://reactjs.org> — Official React.js Documentation
- <https://nodejs.org> — Official Node.js Documentation
- <https://expressjs.com> — Official Express.js Documentation
- <https://mongoosejs.com> — Mongoose ODM Documentation
- <https://jwt.io> — JSON Web Token Documentation
- <https://cloudinary.com> — Cloudinary Documentation
- <https://vitejs.dev> — Vite Build Tool Documentation

2.8.1.5 Overview

This application assists administrators, librarians, and students in an educational institution with managing all library-related tasks. Students can register, look for books, request book issues, request returns, check fines, and handle reservations. Librarians can approve book borrowing and return requests. Administrators have the ability to manage the entire system, which includes books, librarians, members, fines, and reports. The system removes the need for manual paperwork and enables fast, precise, and accessible library management from any device.

2.8.2 Overall Description

The Library Management System is a standalone web application. It features a React.js frontend and a Node.js/Express.js backend that communicate via REST APIs. The database is hosted on MongoDB Atlas in the cloud. User sessions are handled using JWT tokens stored in localStorage. Passwords are secured using bcryptjs.

2.8.2.1 System Interfaces

The client accesses the application through a standard web browser using an internet connection. The frontend interacts with the backend REST API via HTTPS requests using Axios. The backend connects to MongoDB Atlas through a secure cloud connection. Cloudinary is integrated via its API for image upload and retrieval. Nodemailer is used to communicate with Gmail SMTP for sending OTP emails.

2.8.2.2 User Interfaces

Admins and librarians log in through a dedicated login portal (/login-portal) to access their dashboards. Students use the public website, which includes a homepage, book listings, book details, and account pages. All pages are developed with React.js and are fully responsive. Input forms include validation, and errors are displayed using React-Toastify toast notifications.

2.8.2.4 Software Interfaces

Components	Specification
Processor	Intel Pentium IV or equivalent or Higher
RAM	4 GB or more
Hard Disk	10 GB or more
Input Devices	Mouse and Keyboard
Output Devices	Monitor and Printer
Network	Internet Connection Required

Table 2.8.2 Hardware Interfaces

2.8.2.4 Software Interfaces

Component	Technology / Version
Operating System	Windows / macOS / Linux
Web Browser	Google Chrome, Mozilla Firefox, Microsoft Edge
Backend Framework	Node.js with Express.js v4.21
Frontend Framework	React.js v19 with Vite v7
Database	MongoDB Atlas — Mongoose v8.10
Authentication	JSON Web Token — jsonwebtoken v9
Password Encryption	bcryptjs v3
Image Storage	Cloudinary v1.41 with Multer
Email Service	Nodemailer v6.10 using Gmail SMTP
HTTP Client (Frontend)	Axios v1.15
Charts	Chart.js v4.5 with react-chartjs-2

Table 2.8.3 Software Interfaces

2.8.2.5 Communication Interfaces

A stable internet connection is necessary for all users to use the application. The frontend interacts with the backend using HTTPS REST API calls. The backend connects to MongoDB Atlas through a secure cloud connection. Cloudinary is accessed via HTTPS for handling image-related tasks. Nodemailer utilizes Gmail SMTP with TLS encryption to send OTP emails for password reset purposes.

2.8.2.6 Memory Constraints

The system needs a minimum of 4 GB of RAM and 10 GB of hard disk space on the development machine. On the client side, only a modern web browser is needed — no local installation is required. MongoDB Atlas offers 512 MB of cloud storage, which is adequate for the scale of this academic project.

2.8.3 Specific Requirements

2.8.3.1 Functional Requirements

- The system must support three user roles: Admin, Librarian, and Student, each with their own login interface.
- Admin should be able to add, edit, and remove books, including uploading cover images to Cloudinary.
- Admin should have the ability to create and delete librarian accounts and manage student member accounts.
- Admin should be able to set the daily fine rate, maximum fine limit, and grace period for late returns.
- Admin should be able to generate five different types of reports.
- Librarians should be able to view and approve borrow and return requests.
- Students should be able to search for books, request to borrow or return books, see their fines, and manage book reservations.
- The system should automatically calculate fines based on the return date and the fine configuration set by the admin.
- Students should be able to reserve books even when they are not available.
- A 6-digit OTP should be sent to the student's registered email for password reset purposes.

2.8.3.2 Non-Functional Requirements

1. **Reliability:** The system must keep accurate records of all transactions. MongoDB Atlas offers automated backup features. The backend includes a global error handler to avoid unexpected crashes.
2. **Security:** Passwords are encrypted using bcryptjs before being stored in the database. JWT tokens are used to manage user sessions. Role-based middleware (checkRole.js) ensures that only authorized users can access specific routes. OTP verification is required for password reset purposes.
3. **Performance:** A server-side caching utility (cache.js) is used for frequently accessed data like home page statistics, reducing repeated database calls and improving response times.
4. **Scalability:** The MERN stack architecture allows the system to be scaled horizontally. MongoDB Atlas supports scaling with minimal configuration changes. React's lazy loading feature helps reduce initial page load time.

5. **Usability:** The React.js frontend provides a clean and modern interface with smooth animations using Framer Motion. All forms have validation built-in. Success and error messages are shown using React-Toastify. The interface is fully responsive, working well on both desktop and mobile devices.

2.8.3.3 Supporting Technologies

1. **MongoDB:** A NoSQL document database that stores data in flexible JSON-like BSON documents. Unlike relational databases, MongoDB allows each document to have a different structure. MongoDB Atlas is the cloud version used here, offering automated backups, monitoring, and scalable storage.
2. **Express.js:** A lightweight web framework for Node.js that simplifies building REST APIs through routing and middleware. It manages all API routes for users, books, admin, librarian, fines, reservations, and reports. Middlewares like userAuth.js and checkRole.js protect routes and enforce role-based access.
3. **React.js:** A JavaScript library for building fast, interactive user interfaces using a component-based architecture. State management is handled using React Hooks (useState, useEffect). React Router DOM handles client-side navigation, and Axios sends HTTP requests to the backend.
4. **Node.js:** A JavaScript runtime environment built on Chrome's V8 engine. It allows JavaScript to run on the server side using a non-blocking, event-driven model suited for fast and scalable web APIs. It serves as the runtime for the Express.js backend.
5. **JSON Web Token (JWT):** An open standard for securely transmitting information as a digitally signed token. After a successful login, the server generates a JWT token and sends it to the client. The token is stored in localStorage and included in the Authorization header of all API requests. The userAuth middleware verifies the token before allowing access to protected routes.
6. **Cloudinary:** A cloud-based platform for uploading, storing, transforming, and delivering images. Book cover images are uploaded to Cloudinary using Multer and the Multer-Storage-Cloudinary library. Cloudinary returns a URL that is stored in MongoDB and displayed on the frontend.
7. **Nodemailer:** A Node.js module for sending emails from server-side applications. It is set up to use Gmail's SMTP server with credentials stored in environment variables. It sends a 6-digit OTP to the student's registered email for password reset. The OTP is temporarily stored in the MongoDB OTP collection and expires after 10 minutes.

3.System Design

System design is the process of defining the structure, components, modules, interfaces, and data of a system to meet specific requirements. During system design, the overall structure of the software is established. This involves converting the requirements identified in the analysis phase into a detailed plan for building the system.

3.1 Data Design

3.1.1 Database Design

Database design is the process of creating a detailed data model for a database. This model includes all the necessary logical and physical design choices, as well as the parameters for physical storage, which are used to generate a design in a data definition language. This design can then be used to create the actual database. A fully attributed data model includes detailed attributes for each entity.

The database design process starts with gathering requirements and then creating an entity-relationship model. This model is then converted into a relational schema and normalized to eliminate redundancy. The Library Management System uses MongoDB as its database, managed through Mongoose. MongoDB is a NoSQL document database that stores data in flexible JSON-like BSON documents, making it suitable for the semi-structured nature of library data.

There are six major steps in the design process. The first five steps are typically done on paper, and the planning is eventually implemented:

- Identify the tables and their relationships.
- Identify the information needed for each table and relationship.
- Resolve the relationships.
- Verify the design.
- Implement the design.

Applicable Documents

The Database Design refines the System Design document, so the first applicable document is the System Design. The module descriptions provided in the System Requirement Specification are also referenced. Therefore, the second applicable document is the System Requirement Specification.

Normalization

Normalization is typically viewed as a process of applying specific rules to a database design in order to reduce data redundancy. Most textbooks describe this as a three-step procedure, with corresponding normal forms that can be followed in a systematic way.

First Normal Form (1NF)

1NF establishes the basic guidelines for organizing a database:

- Remove duplicate columns from the same table.
- Create separate tables for each group of related data and ensure each row is uniquely identified by a single column or set of columns (the primary key).

Second Normal Form (2NF)

2NF builds on the principles of 1NF by further eliminating redundant data:

- Fulfill all the requirements of 1NF.
- Remove subsets of data that apply to multiple rows in a table and place them in separate tables.
- Establish relationships between these new tables and the original tables using foreign keys.

Third Normal Form (3NF)

3NF takes the process one step further:

- Meet all the requirements of 2NF.
- Eliminate columns that are not dependent on the primary key.

The design of each collection of database collection used in the LMS:

TABLES

1.Collection Name --Users

SI. NO	Field Name	Data Type	Description	Constraint
1	name	String	Full name of the user	Required

2	Email	String	Email address	Required, Unique
3	Password	String	Bcrypt hashed password	Required
4	Role	String	admin / librarian / user	Enum, Default: user
5	Status	String	Active / Inactive	Enum, Default: Active
6	membershipId	String	Auto-generated LIB YYYYMMDD-XXXX	Unique, Sparse
7	Stream	String	Student's academic stream	Required for users
8	Year	Number	Student's academic year	Required for users
9	createdID	String	Account creation timestamp	Auto (timestamps)

Table 3.1 User Collection

2. Collection Name — Books

SI. No	Field Name	Data Type	Description	Constraint
1	Title	String	Title of the book	Required
2	Author	String	Author of the book	Required
3	Category	String	Book category/genre	Required
4	Isbn	String	International Standard Book Number	Required, unique
5	Description	String	Brief description of the book	Required
6	Publisher	String	Publisher name	Default: empty
7	Publication year	Number	Year of publication	Default: null
8	Available copies	Number	Currently available copies	Required
9	totalCopies	Number	Total copies in library	Required
10	Price	Number	Price of the book	Optional
11	coverImage	String	Cloudinary image URL	Optional
12	cloudinaryId	String	Cloudinary image public ID	Default: empty
13	addedBy	ObjectId	Ref to User who added the book	Required

Table 3.2 Books Collections

3. Collection Name — BorrowedBooks

Sl. No	Field Name	Data Type	Description	Constraint
1	bookId	ObjectId	Reference to Book document	Required
2	userId	ObjectId	Reference to User document	Required
3	issueDate	Date	Date the book was issued	Default: now
4	dueDate	Date	Expected return date	Required
5	returnDate	Date	Actual return date	Default: null
6	fineAmount	Number	Fine amount at return time	Default: 0
7	status	String	Requested / Issued / Requested Return / Returned	Enum
8	approvedBy	ObjectId	Ref to Librarian/Admin who approved	Default: null

Table 3.3 BorrowedBooks Collection

4. Collection Name — Fines

Sl. No	Field Name	Data Type	Description	Constraint
1	memberId	ObjectId	Reference to User	Required
2	borrowId	ObjectId	Reference to BorrowedBooks	Required
3	bookId	ObjectId	Reference to Book	Required
4	amount	Number	Total fine amount charged	Required
5	paidStatus	Boolean	Whether fine has been paid	Default: false
6	paidAt	Date	Date of fine payment	Default: null
7	daysOverdue	Number	Number of overdue days	Default: 0
8	ratePerDay	Number	Fine rate at time of return	Default: 10

Table 3.4 Fines Collection

6. Collection Name — FineConfig

Sl. No	Field Name	Data Type	Description	Constraint
1	ratePerDay	Number	Fine charged per overdue day	Default: 10
2	maxFineCap	Number	Maximum fine limit	Default: 500
3	gracePeriod	Number	Days before fine starts	Default: 0
4	updatedBy	ObjectId	Reference to Admin who updated	Optional

Table 3.6 FineConfig Collection

7.Collection Name — OTP

Sl. No	Field Name	Data Type	Description	Constraint
1	email	String	Email address of the user	Required
2	otp	String	6-digit one-time password	Required
3	createdAt	Date	OTP generation time (expires 10 min)	Auto

Table 3.7 OTP Collection

8.Collection Name — Contact

Sl. No	Field Name	Data Type	Description	Constraint
1	name	String	Name of the message sender	Optional
2	email	String	Email of the message sender	Optional
3	subject	String	Subject of the message	Optional
4	message	String	Message content	Optional
5	date	Date	Date of submission	Default: now

Table 3.8 Contact Collection

3.2 System Architectural Design

The Library Management System is developed using the MVC (Model-View-Controller) architecture pattern, implemented through the MERN stack. The MVC pattern divides an application into three main logical components: the Model, the View, and the Controller. Each of these components is responsible for handling specific parts of the application's development.

Model:

The Model is the core part of the MVC pattern. It represents the dynamic data structure of the application and is separate from the user interface. The Model manages the data, logic, and rules of the application. In this system, the Models are defined using Mongoose schemas, such as UserModel, BookModel, BorrowModel, FineModel, and ReservationModel.

View: The View is any way information is displayed, such as a chart, diagram, or table. Different views of the same data can be created. In this system, the View consists of the React.js frontend.

It displays data from the backend API and provides an interactive user interface for Admin, Librarian, and Student users.

Controller:

The Controller handles user input and interacts with the data model. It receives input from the View, may check if it is valid, and then sends it to the Model. In this system, the Controllers are the Express.js route handler functions located in the controller folder.

The Model receives input from the Controller. The View presents the data from the Model to the user. The Controller processes user input and interacts with the Model. Like other software patterns, MVC provides a central solution to a problem while allowing flexibility for different systems.

Service Layer:

Between the Controller and the Model, there is a utility layer. This layer retrieves data from the Model and sends it to the Controller. The service layer helps separate the tasks of data storage (Model), data retrieval (utility), and data manipulation (Controller). In this system, utility files like `fineCalculator.js`, `cache.js`, and `cloudConfig.js` carry out this role.

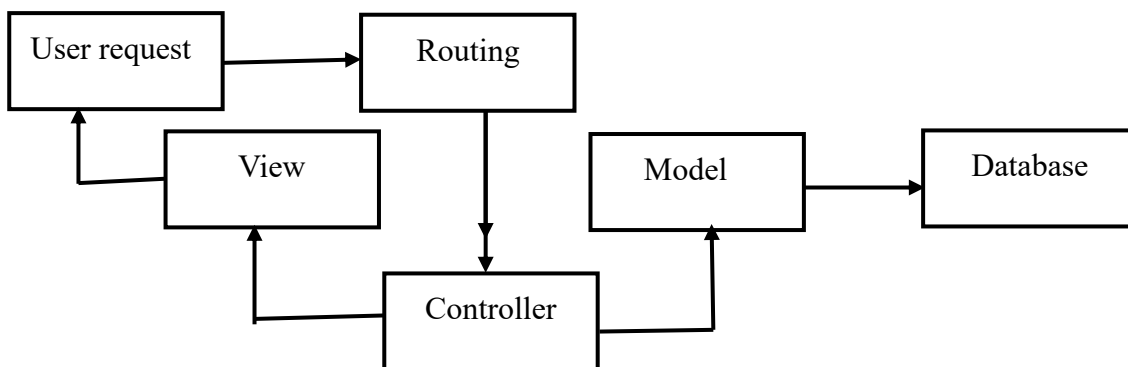


Figure 3.2.1 System Architecture (MVC)

3.3 Component Level Designing

3.3.1 Functional Partitioning

The functional partitioning diagrams show the hierarchical breakdown of each user role's features. Starting from login, each user reaches the main page and can navigate to their respective sections.

Admin Side

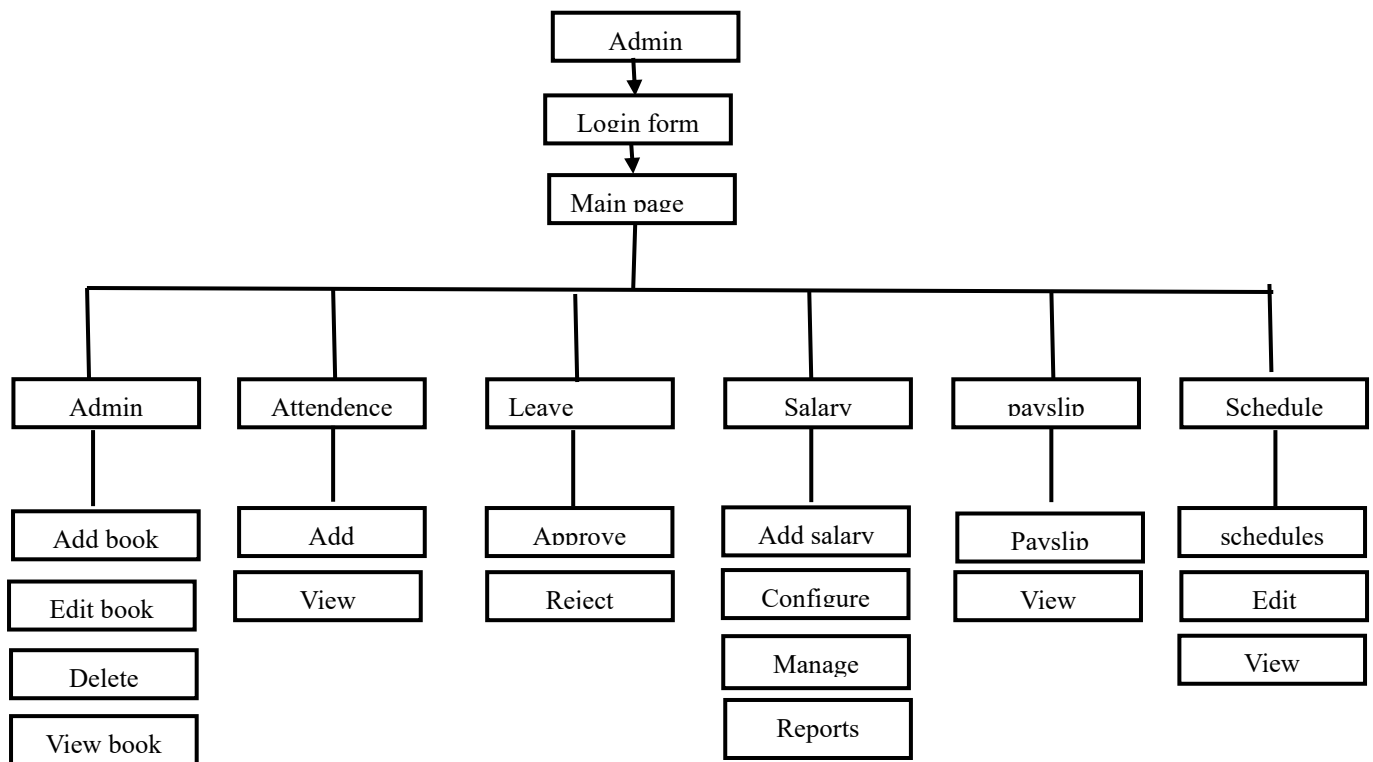


Figure 3.3.1.1 Admin Side Architecture

Student side

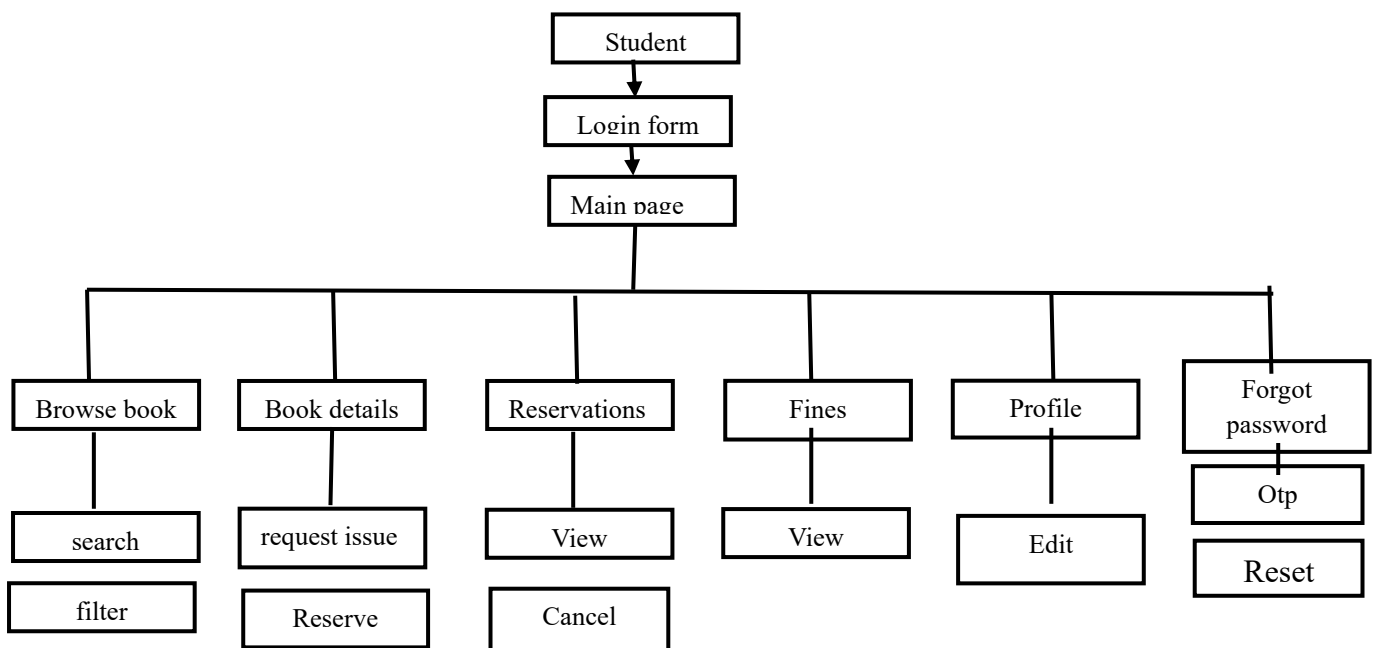


Figure 3.3.1.2 Student Side Architecture

3.3.2 Functional Description of Modules

The Library Management System comprises three primary modules:

- Admin Module
- Librarian Module
- Student (User) Module

Admin Module

- The admin logs in using an email and password. A JWT token is issued upon successful authentication.
- The admin adds a new book with details including title, author, category, ISBN, description, publisher, year, copies, price, and a cover image uploaded via Cloudinary.
- The admin can view the entire book catalog, update any book's information, or permanently delete a book. Deleting a book also removes its image from Cloudinary.
- The admin creates a new librarian account with name, email, and a bcrypt-hashed password.
- The admin manages registered students, toggles their account status between Active and Inactive, and views their individual borrowing history.
- The admin oversees all book reservations made by students across the system.
- The admin reviews all generated fines, marks individual fines as paid, and keeps track of the total collected and pending fine amounts.
- The admin sets the daily fine rate, maximum fine limit, and grace period. These settings apply to all future fine calculations.]
- The admin generates five types of reports: Issued Books, Overdue Books, Fine Collection, Most Borrowed Books, and Member Activity.
- The admin views real-time statistics, including total books, total members, total librarians, number of borrowed books, and a category-wise pie chart.

Librarian Module

- The librarian logs in using email and password. Access to the dashboard is granted only if the role is confirmed as 'librarian'.
- The librarian reviews all pending borrow requests and approves them. Upon approval, the status changes to 'Issued', and the available copies decrease by one.
- The librarian views all currently issued books, including the student's name, book title, issue date, and due date.

- The librarian reviews all return requests and approves them. The return date is recorded, available copies increase by one, and a fine is calculated if the book is overdue.

Student (User) Module

- New students register with their name, email, password, academic stream, and year. A unique membership ID (LIB-YYYYMMDD-XXXX) is automatically generated.
- Students log in and receive a JWT token stored in localStorage for subsequent API requests.
- The home page displays library statistics like total books, categories, and borrowers, along with category-wise book listings.
- Students can browse all books with search options by title, author, or ISBN and filter by category.
- Full book details are displayed along with issue and reservation options based on availability.
- Students can submit borrow requests (with a maximum of four active at a time).
- Students notify the librarian of a return by changing the status to 'Requested Return'.
- Students view and cancel active reservations for unavailable books.
- Students view all fines, including overdue days, amount, and payment status.
- Students can update their profile details, including name, academic stream, and year of study.
- Password reset is initiated via a 6-digit OTP sent to the registered email using Nodemailer.
- Students can submit messages to the library team through a contact form.

3.4 User Interface Design

The user interface for the Library Management System was developed using React.js with Vite as the build tool. Input controls were designed using React's component-based architecture with custom CSS styling. The interface is clean, modern, and fully responsive, functioning well on both desktop and mobile devices.

Technology Environment

The following technologies were used to build the user interface:

React.js:

A JavaScript library for building fast, interactive user interfaces. It uses a component-based architecture where each UI component is reusable. State is managed using React Hooks

like `useState`, `useEffect`, and `useContext`. `React Router DOM` is used for client-side navigation between pages. In this project, `React.js` is used to build all frontend pages for Admin, Librarian, and Student. Lazy loading of components is implemented to reduce initial page load time.

Vite:

A modern frontend build tool that provides an extremely fast development server and optimized production builds. It replaces the traditional Create React App setup with a faster alternative. It supports Hot Module Replacement (HMR), which instantly reflects code changes in the browser during development.

Axios:

A promise-based HTTP client for JavaScript. It is used to make HTTP requests from the React frontend to the Node.js backend REST API. All authenticated API calls include the JWT token in the Authorization header, which is read by the `userAuth` middleware on the backend.

Chart.js with react-chartjs-2:

`Chart.js` is a JavaScript charting library used in the Admin Dashboard to render a Pie chart showing the distribution of books across different categories. The `react-chartjs-2` package provides React wrapper components for `Chart.js`.

Framer Motion:

A production-ready animation library for React. It is used in this project to add smooth page transition animations and component entrance effects, making the user interface feel polished and professional.

React Hook Form:

A performant library for form state management and validation in React.

It is used for all forms in the system, such as Add Book, Registration, Login, and Fine Configuration. It reduces unnecessary re-renders and provides easy-to-implement validation rules.

React-Toastify:

Used to display success and error notifications as toast messages to users. These messages appear as temporary pop-ups at the corner of the screen and automatically disappear after a few seconds.

JWT-decode:

The jwt-decode library is used on the frontend to decode the JWT token stored in localStorage. It extracts user information such as role, name, and user ID from the token payload, used for conditional rendering of UI elements based on the user's role.

Lucide-React and React-Icons:

Provide a large set of SVG-based icons used throughout the interface. These icons improve visual clarity and help users quickly identify actions and features.

4. Software Testing

Introduction

Software testing is an important part of the software development process. It is used to check the correctness, completeness, and quality of the software that has been developed. The main goal of software testing is to find errors in the software. These errors can stop the software from working as the user expects. Errors in software can be broadly divided into three types: requirement errors, design errors, and programming errors.

Software testing involves evaluating a program or system to ensure it meets the required results. Through effective testing, the software can be checked for correctness, completeness, consistency, and compliance with standards. This helps in delivering high-quality software and reduces maintenance costs, leading to better results for users.

4.1 Test Plan

A test plan shows how testing will be carried out. It is a document that explains the objectives, scope, method, and purpose of software testing. This plan outlines the test items, features to be tested, testing tasks, and the people involved. It also specifies the test environment and techniques that will be used. A well-defined test plan is an agreement between testers and users, explaining the role of testing in the software.

A complete test plan helps people outside the testing team understand the reasons and methods for validating the product. An incomplete test plan may fail to check how the software functions on different hardware and operating systems, or when it is used with other software.

Steps in Development of a Test Plan

Carefully created test plans help in conducting effective testing and properly analyzing and preparing error reports. The following steps are followed when developing a test plan:

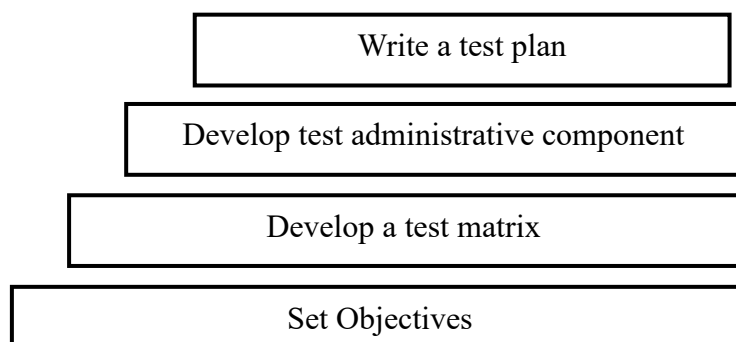


Figure 4.1 Steps in Test Plan

4.2 Test Case Design

A test case is a set of conditions or variables that a tester uses to determine if a system meets the required standards or works correctly. The design of a test case defines the structure of each test case that will be used during testing.

Structure of the test case:

Field	Description
Test Case Id	Display unique test case id
Test Case Name	Display the test case name
Test Objective	Display the test case motivation
Pre-Conditions	Display pre-existing condition of the test case
Expected Result	Display the expected result of the test case
Test Action	Display the action taken for the test case
Actual Result	Display the actual result you get in the test case

Table 4.2.1 Structure of a Test case

Each test case must include the following details:

1. **Test case ID** — A unique number to identify the test case.
2. **Test case name** — A one-line summary of the test objective.
3. **Test case description** — A short explanation of the test objective in two or three lines.
4. **Pre-conditions** — The conditions that must be met before the test is performed.
5. **Expected result** — The output the system or application is expected to produce.
6. **Test description/action** — The action taken during the test.
7. **Actual result** — The result after execution, recorded as Pass or Fail.

4.3 Testing Strategies

To perform testing in a planned and systematic way, a software testing strategy is developed. A testing strategy determines the levels of testing to be used, the methods and techniques, and the tools that will be employed. It also includes decisions about test cases, test specifications, and test case selection.

Creating a testing strategy that meets the project's needs is essential for the success of software development. A software testing strategy should contain full information about the testing procedure, the purpose of testing, and the requirements for testing.

Types of Software Testing Strategies

Different types of software testing strategies are chosen based on the nature and size of the software:

- Analytic testing strategy: Uses both formal and informal techniques to assess and prioritize risks during testing. It takes a full overview of requirements, design, and implementation to determine the aim of testing.
- Model-based testing strategy: Tests software functionality according to real-world scenarios. It identifies the data domains and selects appropriate test cases based on the likelihood of errors in those areas.
- Methodical testing strategy: Tests the functions and status of the software based on a checklist that follows user requirements. It tests functionality, reliability, usability, and performance.
- Process-oriented testing strategy: Tests the software according to existing standards like the IEEE standard. It also checks functionality using automated testing tools.
- Dynamic testing strategy: Tests the software after making collective decisions by the testing team. It also provides information about the software, such as the test cases used to find errors.
- Philosophical testing strategy: Tests the software assuming that components may stop working at any time. It involves the help of software developers, users, and systems analysts.

4.4 Testing Techniques

Testing techniques refer to the different methods followed during testing. There are two main types of techniques used in software testing:

1.Static Testing

In static testing, the program is not executed. It involves informal methods such as code reviews, code reading, and code walkthroughs. Static testing helps identify defects early in the development cycle before actual execution begins.

2. Dynamic Testing

In dynamic testing, the program is executed with test data. The program is traced and debugged. Dynamic testing has two sub-types:

4.5 Different Levels of Testing

4.5.1 Black Box Testing

Black Box Testing is used to check whether input to a function is accepted correctly and output is produced properly. This type of testing examines some aspects of a system without considering the internal structure of the software. Tests are conducted to ensure that every function the product is designed to perform is fully operational.

Errors in the following categories are discovered through black box testing:

- Incorrect or missing functions.
- Interface errors.
- Errors in database structure or external database access.
- Performance errors.
- Initialization and termination errors.

In this project, input to each function is properly accepted and output is correctly produced, confirming the results of black box testing.

4.5.2 White Box Testing

White Box Testing is based on a detailed examination of procedural details. At various points, the status of data management and technical support is tested to see if the expected or declared status matches the actual status. Using this approach, the following test cases are created:

- All logical conditions, both true and false, are tested.
- All loops are executed within their boundaries and operational limits are verified.
- Internal data structures are tested to ensure their validity.

4.5.3 Unit Testing

Unit testing is a process used to check if a specific part of the source code is working correctly. Test cases are created for every function and method so that any changes causing a regression can be quickly found and fixed.

Benefits of Unit Testing

The main purpose of unit testing is to check each part of the program separately to ensure it works as intended. It gives a clear set of rules that the code must follow:

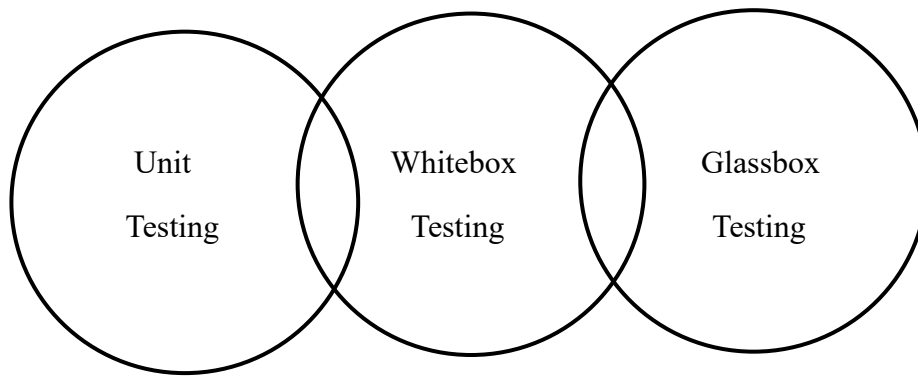


Figure 4.2 Level of Testing

- **Enables change** — Unit testing allows developers to modify code later without worrying about breaking the module.
- **Makes integration easier** — By ensuring each code unit works correctly, unit testing makes the integration testing process simpler.
- **Acts as documentation** — Unit tests explain what each module is supposed to do.
- **Keeps interface and implementation separate** — Since each function is tested individually, changes to the code's inner workings do not affect the test interface.

4.5.4 Integrated System Testing

Integrated System Testing (IST) is a method used to check the overall structure of the software while testing for issues that occur at the interfaces between different parts. The goal is to test the whole system based on the design, using unit-tested modules. IST can be done in two main ways: top-down integration or bottom-up integration.

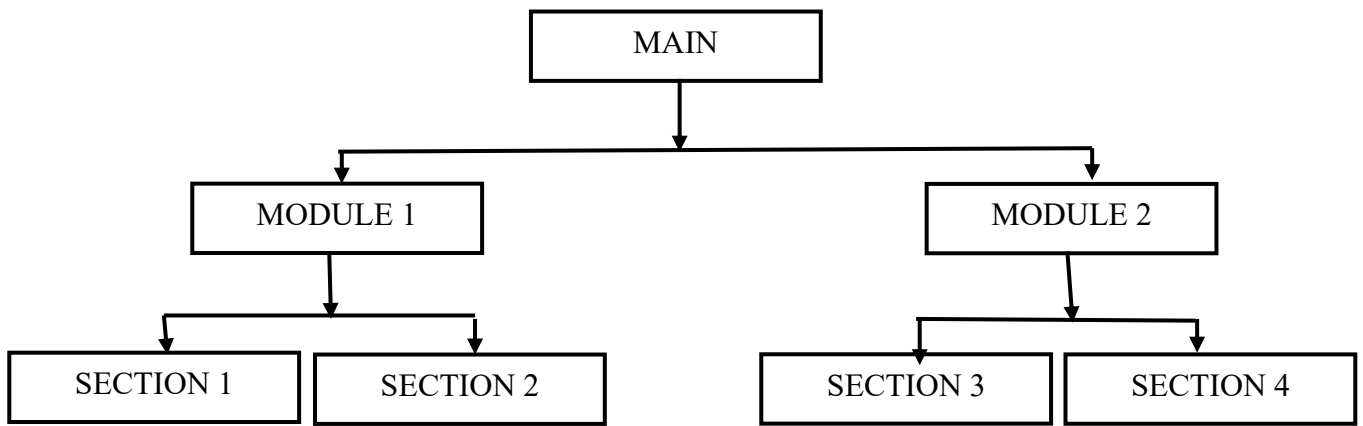


Figure 4.3 Integrated System Testing

4.5.5 System Testing

In system testing, the entire system is tested using real data. The aim is to make sure the system meets the client's requirements. System testing typically has three stages:

- **Alpha Testing:** The system is tested by developers in a controlled environment.
- **Beta Testing:** The system is tested by users who are not part of the development team before the final release.
- **Acceptance Testing:** The system is tested by the actual users based on agreed-upon criteria.

4.5.6 User Acceptance Testing

User Acceptance Testing (UAT) is carried out by users or on their behalf to confirm that the software performs as described in the Business Requirement Document. UAT ensures the following:

- All functional requirements are met.
- All performance requirements are achieved.
- Other requirements like transportability, compatibility, and error recovery are fulfilled.
- The acceptance criteria set by the users are satisfied.

4.6 Software Test Results

The following test cases were carried out to check the functionality of all major features of the Library Management System. All the test cases were successful and passed.

Test Case Id	1
Test Case Name	Admin Login
Test Objective	To verify that the admin login function works correctly
Pre-Conditions	Admin login portal must be accessible
Expected Result	Admin should be authenticated and redirected to the dashboard
Test Action	Enter valid admin email and password and click login
Actual Result	Pass

Test Case Id	2
Test Case Name	Add Book
Test Objective	To verify that a new book can be added with all required details
Pre-Conditions	Admin must be logged in; Add Book form must be open
Expected Result	Book should be saved to MongoDB and appear in the book list with Cloudinary image URL
Test Action	Fill in title, author, category, ISBN, description, copies, and upload cover image
Actual Result	Pass

Test Case Id	3
Test Case Name	Edit Book
Test Objective	To verify that book details can be updated by the admin
Pre-Conditions	Admin must be logged in; existing book must exist in the system
Expected Result	Book details should be updated in MongoDB and reflected in the view immediately
Test Action	Click edit on an existing book, modify the required fields, and submit
Actual Result	Pass

Test Case Id	4
Test Case Name	Delete Book
Test Objective	To verify that a book can be permanently deleted from the system
Pre-Conditions	Admin must be logged in; book must not have active borrow records
Expected Result	Book should be removed from MongoDB and its Cloudinary image should be deleted
Test Action	Click delete on an existing book and confirm the action
Actual Result	Pass

Test Case Id	5
Test Case Name	Add Librarian
Test Objective	To verify that a new librarian account can be created by admin
Pre-Conditions	Admin must be logged in; Add Librarian form must be open
Expected Result	New librarian account should be created with bcrypt-hashed password in MongoDB
Test Action	Enter librarian name, email, and password and submit the form
Actual Result	Pass

Test Case Id	6
Test Case Name	Student Registration
Test Objective	To verify that a new student can register successfully
Pre-Conditions	Registration page must be accessible
Expected Result	New user account should be created with auto-generated membership ID (LIB-YYYYMMDD-XXXX)
Test Action	Enter name, email, password, stream, and year and submit the registration form
Actual Result	Pass

Test Case Id	7
Test Case Name	Student Login
Test Objective	To verify that a registered student can log in successfully
Pre-Conditions	Student login page must be accessible
Expected Result	Student should be authenticated and receive a JWT token stored in localStorage
Test Action	Enter valid registered email and password and click login
Actual Result	Pass

Test Case Id	8
Test Case Name	Request Book Issue
Test Objective	To verify that a student can submit a borrow request
Pre-Conditions	Student must be logged in; selected book must have available copies
Expected Result	A borrow record with status 'Requested' should be created in BorrowedBooks collection
Test Action	Navigate to book details page and click the Request button
Actual Result	Pass

Test Case Id	9
Test Case Name	Approve Issue Request
Test Objective	To verify that a librarian can approve a pending borrow request
Pre-Conditions	Librarian must be logged in; borrow request with status 'Requested' must exist
Expected Result	Status changes to 'Issued', availableCopies decreases by 1, issueDate and dueDate set
Test Action	Click Approve on the Issue Requests page
Actual Result	Pass

Test Case Id	10
Test Case Name	Request Book Return
Test Objective	To verify that a student can submit a return request
Pre-Conditions	Student must be logged in; book must have status 'Issued'
Expected Result	Borrow record status should change to 'Requested Return'
Test Action	Navigate to currently issued books and click Request Return
Actual Result	Pass

Test Case Id	11
Test Case Name	Approve Return Request
Test Objective	To verify that a librarian can approve a book return
Pre-Conditions	Librarian must be logged in; return request with status 'Requested Return' must exist
Expected Result	Status changes to 'Returned', returnDate set, availableCopies increases by 1, fine calculated if overdue
Test Action	Click Approve on the Return Requests page
Actual Result	Pass

Test Case Id	12
Test Case Name	Fine Configuration
Test Objective	To verify that admin can update fine settings
Pre-Conditions	Admin must be logged in; Fine Configuration page must be open
Expected Result	Fine rate, maximum cap, and grace period should be saved in FineConfig collection
Test Action	Enter new values for ratePerDay, maxFineCap, gracePeriod and save
Actual Result	Pass

Test Case Id	13
Test Case Name	Reserve Book
Test Objective	To verify that a student can reserve a book with no available copies
Pre-Conditions	Student must be logged in; selected book must have 0 available copies
Expected Result	A new Reservation record with status 'Pending' should be created
Test Action	Click Reserve on a book details page showing no available copies
Actual Result	Pass

Test Case Id	14
Test Case Name	Cancel Reservation
Test Objective	To verify that a student can cancel an active reservation
Pre-Conditions	Student must be logged in; reservation must have status 'Pending' or 'Notified'
Expected Result	Reservation status should change to 'Cancelled'
Test Action	Navigate to My Reservations page and click Cancel
Actual Result	Pass

Test Case Id	15
Test Case Name	View My Fines
Test Objective	To verify that a student can view their fine records
Pre-Conditions	Student must be logged in; at least one fine record must exist for the student
Expected Result	List of fines with book details, overdue days, amount, and paid status should display
Test Action	Navigate to My Fines page
Actual Result	Pass

Test Case Id	16
Test Case Name	Forgot Password OTP
Test Objective	To verify the complete OTP-based password reset flow
Pre-Conditions	User must have a registered email; Nodemailer must be configured
Expected Result	New password should be saved; old password should no longer work
Test Action	Enter email, receive 6-digit OTP via email, verify OTP, enter new password and submit
Actual Result	Pass

Test Case Id	17
Test Case Name	Generate Report
Test Objective	To verify that admin can generate an overdue books report
Pre-Conditions	Admin must be logged in; at least one book must be past its due date
Expected Result	List of overdue borrow records with days overdue and fine amount should display
Test Action	Navigate to Reports page and click Overdue Books report
Actual Result	Pass

Test Case Id	18
Test Case Name	Toggle Member Status
Test Objective	To verify that admin can activate or deactivate a student account
Pre-Conditions	Admin must be logged in; student account must exist in the system
Expected Result	Student account status should toggle between Active and Inactive
Test Action	Click Toggle Status button on the Manage Members page
Actual Result	Pass

Test Case Id	19
Test Case Name	Mark Fine as Paid
Test Objective	To verify that admin can mark a fine as paid
Pre-Conditions	Admin must be logged in; an unpaid fine must exist in the system
Expected Result	Fine paidStatus should change to true and paidAt date should be recorded
Test Action	Click Mark as Paid on the Fine Management page
Actual Result	Pass

Test Case Id	20
Test Case Name	Logout
Test Objective	To verify that the logout function works correctly for all user types
Pre-Conditions	User must be currently logged in with a valid JWT token
Expected Result	JWT token should be removed from localStorage and user redirected to login page
Test Action	Click the Logout button in the navigation menu
Actual Result	Pass

Test Case Id	21
Test Case Name	Librarian Login
Test Objective	To verify that a librarian can log in successfully using valid credentials
Pre-Conditions	Librarian login portal (/login-portal) must be accessible; librarian account must exist
Expected Result	Librarian should be authenticated, role verified as 'librarian', and redirected to the librarian dashboard
Test Action	Enter valid librarian email and password and click Login
Actual Result	Pass

Test Case Id	22
Test Case Name	View Pending Issue Requests
Test Objective	To verify that a librarian can view all pending borrow requests submitted by students
Pre-Conditions	Librarian must be logged in; at least one borrow request with status 'Requested' must exist
Expected Result	List of all pending borrow requests with student name, book title, and request date should display
Test Action	Navigate to the Issue Requests page in the librarian dashboard
Actual Result	Pass

Test Case Id	23
Test Case Name	Approve Borrow Request
Test Objective	To verify that a librarian can approve a student's pending borrow request
Pre-Conditions	Librarian must be logged in; at least one borrow request with status 'Requested' must exist
Expected Result	Status changes to 'Issued', availableCopies decreases by 1, issueDate and dueDate are set
Test Action	Click Approve on a pending request in the Issue Requests page
Actual Result	Pass

Test Case Id	24
Test Case Name	View Issued Books
Test Objective	To verify that a librarian can view the complete list of all currently issued books
Pre-Conditions	Librarian must be logged in; at least one book must have borrow status 'Issued'
Expected Result	List of all issued books with student name, book title, issue date, and due date should display
Test Action	Navigate to the Books Borrowed page in the librarian dashboard
Actual Result	Pass

Test Case Id	25
Test Case Name	View Pending Return Requests
Test Objective	To verify that a librarian can view all return requests submitted by students
Pre-Conditions	Librarian must be logged in; at least one borrow record with status 'Requested Return' must exist
Expected Result	List of all return requests with student name, book title, issue date, and due date should display
Test Action	Navigate to the Return Requests page in the librarian dashboard
Actual Result	Pass

Test Case Id	26
Test Case Name	Approve Return Request
Test Objective	To verify that a librarian can approve a book return and that fines are calculated for overdue books
Pre-Conditions	Librarian must be logged in; a return request with status 'Requested Return' must exist; book must be past its due date
Expected Result	Status changes to 'Returned', returnDate is recorded, availableCopies increases by 1, fine is calculated based on FineConfig if overdue
Test Action	Click Approve on a return request in the Return Requests page
Actual Result	Pass

5. Conclusion and Future Scope

Conclusion

The Library Management System has been successfully built and tested. The software requirements specification documents are used to validate the product. It satisfies all the requirements and all the project objectives are met. The system is thoroughly tested for bugs and any kind of errors and all identified issues have been resolved to make the system fault free.

The salient features of the system are:

- The system is user-friendly with separate login portals for Admin, Librarian, and Student, allowing each user to access only the features relevant to their role.
- MongoDB Atlas has successfully replaced manual paper-based registers. All records including books, borrow history, fines, and reservations are stored securely in the cloud.
- Fine calculation is fully automated based on configurable rules — the daily rate, maximum fine cap, and grace period can all be set by the admin, eliminating manual disputes.
- JWT-based authentication and bcrypt password hashing ensure secure access. Role-based middleware guarantees that each user can only reach features permitted for their role.
- The admin can generate five types of reports — Issued Books, Overdue Books, Fine Collection, Most Borrowed Books, and Member Activity — with a single click from the dashboard.
- Students can browse books, request issues, submit return requests, manage reservations, and view their fines from any device through a web browser at any time.

Future Scope:

The Library Management System can be enhanced in future by including many additional features and improvements such as:

- A mobile application for Android and iOS can be developed using React Native, allowing students and staff to access the system on smartphones with a native app experience.
- An automated email reminder system can be added using Nodemailer to notify students before and after their book due dates, helping to reduce overdue returns.
- A barcode or QR code scanning feature can be integrated to allow librarians to scan books directly during issue and return, instead of searching manually.
- The reservation system can be extended with automatic expiry notifications so that when a reserved book becomes available, the student is notified by email automatically.
- The system can be scaled to support multiple library branches within the same institution, with a centralized book inventory and separate dashboards for each branch.
- An AI-based book recommendation module can be added to suggest books to students based on their borrowing history, academic stream, and year of study.